

GUÍA DE FUNCIONES BUILT-IN DE PYTHON

Explicación y ejemplos prácticos de los built-ins disponibles sin importación

Python 3.14 Referencia práctica	71 nombres incluidos en la referencia oficial
Enfoque qué hace · cuándo usarlo · ejemplo	Seguridad aviso para eval(), exec() y código dinámico

Cómo leer esta guía. Cada ficha contiene la firma habitual, una explicación breve y un ejemplo ejecutable. La lista sigue la referencia oficial de Python 3.14.7. La documentación incluye tanto funciones como clases built-in invocables (por ejemplo `int()`, `list()` y `dict()`), por lo que se incluyen todas para que la referencia sea completa.

Fuente de referencia: documentación oficial de Python 3.14.7, sección “Built-in Functions”. Consulta la documentación oficial para detalles de comportamiento, excepciones y cambios de versión.

Mapa rápido: ¿qué built-in necesito?

Categoría	Funciones / clases destacadas
Números y conversiones	abs, bin, bool, complex, float, hex, int, oct, ord, pow, round
Iteración y colecciones	aiter, anext, all, any, enumerate, filter, iter, len, map, max, min, next, range, reversed, sorted, sum, zip
Texto y representación	ascii, chr, format, repr, str
Objetos y atributos	callable, delattr, dir, getattr, hasattr, hash, id, isinstance, issubclass, setattr, type, vars
Estructuras y datos binarios	bytearray, bytes, dict, frozenset, list, memoryview, object, set, slice, tuple
Clases y orientación a objetos	classmethod, property, staticmethod, super
Entrada, salida y ficheros	input, open, print
Entorno y depuración	breakpoint, globals, help, locals
Código dinámico / importación	compile, eval, exec, __import__

Consejos esenciales

- Preferencia: usa construcciones idiomáticas de Python antes que soluciones dinámicas innecesarias.
- Iteración: `enumerate()`, `zip()`, `map()`, `filter()` y comprehensions permiten expresar transformaciones de forma clara.
- Seguridad: no ejecutes texto no confiable con `eval()` o `exec()`.
- Ficheros: prefiere `with open(..., encoding="utf-8") as f:` para cerrar automáticamente.
- Tipos: `isinstance()` suele ser más flexible que comparar `type(x)` exactamente.

1. abs()

Firma: `abs(x)`

Devuelve el valor absoluto de un número. En complejos, devuelve su módulo.

```
abs(-7)           # 7
abs(3.5)          # 3.5
abs(3+4j)         # 5.0
```

2. aiter()

Firma: `aiter(async_iterable)`

Obtiene el iterador asíncrono de un objeto iterable asíncrono. Es la versión asíncrona de `iter()`.

```
async def demo(agen):
    it = aiter(agen)
    print(await anext(it))
```

3. all()

Firma: `all(iterable)`

Devuelve `True` si todos los elementos son verdaderos. Un iterable vacío produce `True`.

```
all([True, 1, "ok"]) # True
all([True, 0, 1])    # False
```

4. `anext()`

Firma: `anext(async_iterator[, default])`

Obtiene el siguiente elemento de un iterador asíncrono; con `default` evita `StopAsyncIteration`.

```
async def demo(it):
    valor = await anext(it, "fin")
    print(valor)
```

5. `any()`

Firma: `any(iterable)`

Devuelve `True` si al menos un elemento es verdadero. Un iterable vacío produce `False`.

```
any([0, "", 4])      # True
any([0, False, None]) # False
```

6. `ascii()`

Firma: `ascii(obj)`

Devuelve una representación similar a `repr()`, escapando caracteres no ASCII.

```
ascii("España")      # 'Espa\u00f1a'
```

7. bin()

Firma: bin(x)

Convierte un entero a una cadena con su representación binaria y prefijo 0b.

```
bin(10)          # '0b1010'
format(10, 'b')  # '1010'
```

8. bool()

Firma: bool(x=False)

Convierte un valor a booleano mediante las reglas de verdad de Python. Técnicamente es una clase built-in.

```
bool(0)          # False
bool("Python")   # True
```

9. breakpoint()

Firma: breakpoint(*args, **kwargs)

Pausa la ejecución y entra en el depurador configurado por Python. Útil durante el desarrollo.

```
x = 10
breakpoint()      # inspeccionar x
print(x)
```

10. bytearray()

Firma: `bytearray(source=b'', encoding=..., errors=...)`

Crea una secuencia mutable de bytes. Es útil para manipular datos binarios modificables.

```
datos = bytearray(b"ABC")
datos[0] = 90
print(datos)           # bytearray(b'ZBC')
```

11. bytes()

Firma: `bytes(source=b'', encoding=..., errors=...)`

Crea una secuencia inmutable de bytes, útil para datos binarios o codificados.

```
datos = bytes("Hola", "utf-8")
print(datos)           # b'Hola'
```

12. callable()

Firma: `callable(obj)`

Indica si un objeto parece invocable, por ejemplo una función, clase o instancia con `__call__()`.

```
callable(len)          # True
callable(42)           # False
```

13. chr()

Firma: chr(codepoint)

Devuelve el carácter Unicode correspondiente a un punto de código.

```
chr(65)           # 'A'
chr(8364)         # '€'
```

14. classmethod()

Firma: classmethod(func)

Convierte un método en método de clase: recibe la clase (cls) como primer argumento implícito.

```
class Persona:
    @classmethod
    def crear(cls):
        return cls()
```

15. compile()

Firma: compile(source, filename, mode, ...)

Compila código Python a un objeto código o AST. mode suele ser 'exec', 'eval' o 'single'.

```
codigo = compile("2 + 3", "<string>", "eval")
print(eval(codigo))    # 5
```

16. complex()

Firma: `complex(real=0, imag=0)`

Crea o convierte un número complejo. Es una clase built-in.

```
z = complex(2, 3)
print(z)           # (2+3j)
print(z.real)      # 2.0
```

17. delattr()

Firma: `delattr(obj, name)`

Elimina dinámicamente un atributo de un objeto; equivale conceptualmente a `del obj.name`.

```
class A: pass
a = A()
a.x = 10
delattr(a, "x")
```

18. dict()

Firma: `dict(...)`

Crea un diccionario de pares clave-valor. Las claves deben ser hashables.

```
persona = dict(nombre="Ana", edad=25)
print(persona["nombre"]) # Ana
```

19. dir()

Firma: `dir([obj])`

Lista nombres de atributos disponibles. Sin argumento, muestra nombres del ámbito local.

```
print(dir(str))
texto = "hola"
print("upper" in dir(texto)) # True
```

20. divmod()

Firma: `divmod(a, b)`

Devuelve simultáneamente cociente y resto de la división entera.

```
q, r = divmod(17, 5)
print(q, r)          # 3 2
```

21. enumerate()

Firma: `enumerate(iterable, start=0)`

Itera produciendo pares (índice, elemento). Evita llevar un contador manual.

```
for i, fruta in enumerate(["pera", "uva"], start=1):
    print(i, fruta)
```

22. eval()

Firma: `eval(source, globals=None, locals=None)`

Evalúa una expresión Python y devuelve su resultado. No debe usarse con entrada no confiable.

```
expresion = "10 * 2"
resultado = eval(expresion) # 20
```

23. exec()

Firma: `exec(source, globals=None, locals=None, *, closure=None)`

Ejecuta código Python dinámico. Es potente pero peligroso con código no confiable.

```
namespace = {}
exec("x = 10", namespace)
print(namespace["x"]) # 10
```

24. filter()

Firma: `filter(function, iterable)`

Crea un iterador con los elementos para los que la función devuelve un valor verdadero.

```
pares = filter(lambda x: x % 2 == 0, range(6))
print(list(pares)) # [0, 2, 4]
```

25. float()

Firma: `float(x=0.0)`

Convierte un valor a número de coma flotante. Es una clase built-in.

```
float("3.14")      # 3.14
float(5)            # 5.0
```

26. format()

Firma: `format(value, spec='')`

Obtiene la representación formateada de un valor según una especificación de formato.

```
format(3.14159, ".2f") # '3.14'
format(255, "#x")      # '0xff'
```

27. frozenset()

Firma: `frozenset(iterable=())`

Crea un conjunto inmutable y hashable, útil como clave de diccionario o elemento de otro conjunto.

```
colores = frozenset(["rojo", "azul"])
print("rojo" in colores) # True
```

28. getattr()

Firma: `getattr(obj, name[, default])`

Obtiene dinámicamente un atributo por nombre; puede devolver un valor por defecto si no existe.

```
class A: x = 10
a = A()
print(getattr(a, "x")) # 10
print(getattr(a, "y", 0)) # 0
```

29. globals()

Firma: `globals()`

Devuelve el diccionario que representa el espacio de nombres global actual.

```
x = 10
print(globals()["x"]) # 10
```

30. hasattr()

Firma: `hasattr(obj, name)`

Devuelve True si el objeto tiene un atributo con ese nombre y puede recuperarse sin `AttributeError`.

```
class A: x = 1
a = A()
print(hasattr(a, "x")) # True
```

31. hash()

Firma: `hash(obj)`

Devuelve el valor hash de un objeto hashable, usado por estructuras como dict y set.

```
print(hash("Python"))
print(hash((1, 2)))
```

32. help()

Firma: `help([obj])`

Abre el sistema de ayuda interactivo. Es especialmente útil en la consola para explorar APIs.

```
help(str.upper)
help("list")
```

33. hex()

Firma: `hex(x)`

Convierte un entero a una cadena hexadecimal con prefijo 0x.

```
hex(255)           # '0xff'
format(255, 'x')   # 'ff'
```

34. id()

Firma: `id(obj)`

Devuelve un entero que identifica al objeto durante su vida. No debe usarse como identificador persistente.

```
a = []  
b = a  
print(id(a) == id(b)) # True
```

35. input()

Firma: `input(prompt='')`

Lee una línea de texto desde la entrada estándar y devuelve una cadena, sin el salto final.

```
nombre = input("Nombre: ")  
print("Hola", nombre)
```

36. int()

Firma: `int(x=0, base=10)`

Convierte a entero o crea un entero. Con base, interpreta una cadena según esa base.

```
int("101", 2)      # 5  
int(3.9)           # 3
```

37. isinstance()

Firma: `isinstance(obj, class_or_tuple)`

Comprueba si un objeto es instancia de una clase o de alguna clase de una tupla.

```
valor = 10
print(isinstance(valor, int)) # True
```

38. isinstance()

Firma: `issubclass(cls, class_or_tuple)`

Comprueba si una clase es subclase de otra, directa o indirectamente.

```
class Animal: pass
class Perro(Animal): pass
print(issubclass(Perro, Animal)) # True
```

39. iter()

Firma: `iter(iterable) / iter(callable, sentinel)`

Obtiene un iterador. Con callable + sentinel, llama repetidamente hasta obtener el valor centinela.

```
it = iter([10, 20, 30])
print(next(it))          # 10
```

40. len()

Firma: len(obj)

Devuelve el número de elementos de un objeto que implementa el protocolo de tamaño.

```
len("Python")          # 6
len([10, 20, 30])      # 3
```

41. list()

Firma: list(iterable=())

Crea una lista mutable. Puede convertir cualquier iterable en una lista.

```
lista = list(range(3))
print(lista)          # [0, 1, 2]
```

42. locals()

Firma: locals()

Devuelve un mapeo que representa el espacio de nombres local actual. Su comportamiento de escritura depende del contexto.

```
x = 10
print(locals()["x"]) # 10
```

43. map()

Firma: `map(function, iterable, ...)`

Crea un iterador aplicando una función a los elementos de uno o varios iterables.

```
cuadrados = map(lambda x: x*x, [1, 2, 3])
print(list(cuadrados))    # [1, 4, 9]
```

44. max()

Firma: `max(iterable, *, key=None, default=...) / max(a, b, ...)`

Devuelve el mayor elemento. `key` permite definir qué criterio se compara.

```
max([3, 9, 2])           # 9
max("Python", key=len)   # 'Python'
```

45. memoryview()

Firma: `memoryview(obj)`

Crea una vista sobre el búfer de otro objeto, evitando copias al trabajar con datos binarios.

```
datos = bytearray(b"abcdef")
v = memoryview(datos)
v[0] = ord('Z')
print(datos)           # bytearray(b'Zbcdef')
```

46. min()

Firma: `min(iterable, *, key=None, default=...) / min(a, b, ...)`

Devuelve el menor elemento; key permite especificar el criterio de comparación.

```
min([3, 9, 2])           # 2
min(["aaa", "b"], key=len) # 'b'
```

47. next()

Firma: `next(iterator[, default])`

Obtiene el siguiente elemento de un iterador. Con default, evita StopIteration cuando se agota.

```
it = iter([10, 20])
print(next(it))      # 10
print(next(it, None)) # 20
```

48. object()

Firma: `object()`

Crea el objeto base de Python, raíz de la jerarquía de clases. Normalmente se usa como clase base.

```
class Persona(object):
    pass
p = Persona()
```

49. oct()

Firma: oct(x)

Convierte un entero a una cadena con representación octal y prefijo 0o.

```
oct(8)           # '0o10'
format(8, 'o')   # '10'
```

50. open()

Firma: open(file, mode='r', ...)

Abre un fichero y devuelve un objeto de archivo. Es fundamental para lectura y escritura.

```
with open("datos.txt", "r", encoding="utf-8") as f:
    texto = f.read()
print(texto)
```

51. ord()

Firma: ord(c)

Devuelve el punto de código Unicode de un carácter de longitud 1. Es la operación inversa de chr().

```
ord('A')         # 65
ord('€')         # 8364
```

52. pow()

Firma: pow(base, exp[, mod])

Calcula base elevada a exp. Con mod, calcula potencia modular de forma eficiente.

```
pow(2, 10)           # 1024
pow(2, 10, 1000)     # 24
```

53. print()

Firma: print(*objects, sep=' ', end='\n', file=None, flush=False)

Escribe objetos en un flujo de salida, normalmente la consola. Permite controlar separador, final y destino.

```
print("Hola", "Python", sep=" - ")
print("listo", end="!\n")
```

54. property()

Firma: property(fget=None, fset=None, fdel=None, doc=None)

Crea un atributo administrado mediante métodos getter/setter/deleter. Muy usado en clases.

```
class Persona:
    def __init__(self, edad): self._edad = edad
    @property
    def edad(self): return self._edad
```

55. range()

Firma: range(stop) / range(start, stop[, step])

Representa una secuencia aritmética eficiente en memoria, muy usada en bucles.

```
for i in range(2, 8, 2):  
    print(i)                # 2, 4, 6
```

56. repr()

Firma: repr(obj)

Devuelve una representación textual orientada a depuración y, cuando es posible, no ambigua.

```
texto = "hola\n"  
print(repr(texto))          # 'hola\n'
```

57. reversed()

Firma: reversed(seq)

Devuelve un iterador que recorre una secuencia en orden inverso.

```
print(list(reversed([1, 2, 3]))) # [3, 2, 1]
```

58. round()

Firma: round(number, ndigits=None)

Redondea un número. Con ndigits se especifica el número de decimales (o posición decimal).

```
round(3.14159, 2)      # 3.14
round(1234, -2)        # 1200
```

59. set()

Firma: set(iterable=())

Crea un conjunto mutable de elementos únicos y hashables. Es útil para pertenencia y operaciones de conjuntos.

```
nums = set([1, 2, 2, 3])
print(nums)          # {1, 2, 3}
```

60. setattr()

Firma: setattr(obj, name, value)

Asigna dinámicamente un atributo a un objeto; complementa getattr() y delattr().

```
class A: pass
a = A()
setattr(a, "nombre", "Ana")
print(a.nombre)
```

61. slice()

Firma: slice(stop) / slice(start, stop[, step])

Crea un objeto que representa una operación de corte (slicing), útil para reutilizar índices.

```
s = slice(1, 5, 2)
print([0,1,2,3,4,5][s]) # [1, 3]
```

62. sorted()

Firma: sorted(iterable, *, key=None, reverse=False)

Devuelve una nueva lista ordenada. No modifica el iterable original.

```
datos = [3, 1, 2]
print(sorted(datos))      # [1, 2, 3]
print(sorted(datos, reverse=True))
```

63. staticmethod()

Firma: staticmethod(func)

Convierte un método en método estático: no recibe automáticamente ni self ni cls.

```
class Calculadora:
    @staticmethod
    def suma(a, b):
        return a + b
print(Calculadora.suma(2, 3))
```

64. str()

Firma: `str(object='')`

Convierte un objeto en texto legible o crea una cadena. Es una clase built-in.

```
str(123)           # '123'
str([1, 2])        # '[1, 2]'
```

65. sum()

Firma: `sum(iterable, /, start=0)`

Suma los elementos de un iterable empezando por start. Para grandes cálculos numéricos puede convenir `math.fsum()`.

```
sum([1, 2, 3])      # 6
sum([1, 2], 10)     # 13
```

66. super()

Firma: `super([type[, object-or-type]])`

Devuelve un proxy para acceder a métodos y atributos de una clase base siguiendo el MRO.

```
class Hijo(Padre):
    def saludo(self):
        super().saludo()
```

67. tuple()

Firma: tuple(iterable=())

Crea una tupla, secuencia ordenada e inmutable.

```
t = tuple([1, 2, 3])
print(t)                # (1, 2, 3)
```

68. type()

Firma: type(obj) / type(name, bases, namespace)

Devuelve el tipo de un objeto o, con tres argumentos, crea dinámicamente una clase.

```
type(42) is int          # True
Clase = type("A", (), {})
```

69. vars()

Firma: vars([obj])

Devuelve `__dict__` del objeto cuando está disponible; sin argumento, equivale al espacio de nombres local.

```
class Persona:
    def __init__(self): self.nombre = "Ana"
p = Persona()
print(vars(p))          # {'nombre': 'Ana'}
```

70. zip()

Firma: `zip(*iterables, strict=False)`

Agrupar elementos por posición y crea un iterador de tuplas. `strict=True` detecta longitudes desiguales.

```
nombres = ["Ana", "Luis"]
edades = [25, 31]
print(list(zip(nombres, edades))) # [('Ana', 25), ('Luis', 31)]
```

71. __import__()

Firma: `__import__(name, globals=None, locals=None, fromlist=(), level=0)`

Implementa el mecanismo interno de importación. Normalmente se prefiere usar `import` o `importlib`.

```
math = __import__("math")
print(math.sqrt(9)) # 3.0
```

Patrones prácticos para combinar built-ins

Filtrar y transformar

```
numeros = range(1, 11)
pares = filter(lambda x: x % 2 == 0, numeros)
cuadrados = map(lambda x: x*x, pares)
print(list(cuadrados)) # [4, 16, 36, 64, 100]
```

Enumerar y agrupar

```
nombres = ["Ana", "Luis"]
edades = [25, 31]

for i, (nombre, edad) in enumerate(zip(nombres, edades), start=1):
    print(i, nombre, edad)
```

Ordenar con criterio

```
personas = [
    {"nombre": "Ana", "edad": 31},
    {"nombre": "Luis", "edad": 25},
]
ordenadas = sorted(personas, key=lambda p: p["edad"])
print(ordenadas)
```

Lectura segura de texto

```
with open("datos.txt", "r", encoding="utf-8") as f:
    for linea in f:
        print(linea.rstrip())
```

Conversión y validación

```
dato = "42"
if dato.isdigit():
    numero = int(dato)
    print(numero + 8) # 50
```

Avisos de seguridad y diseño

`eval()` y `exec()`. Ambas funciones pueden ejecutar código arbitrario. Nunca pases directamente entrada de usuarios o datos externos sin confianza. Si necesitas interpretar datos con sintaxis limitada, considera alternativas específicas como `ast.literal_eval()` cuando corresponda.

`__import__()`. Es parte del mecanismo interno de importación. Para código normal es preferible usar `import` o APIs de `importlib`.

`globals()` y `locals()`. Son herramientas introspectivas. Modificar los diccionarios devueltos no debe tratarse como una forma general de crear o cambiar variables locales.

Referencia oficial

Python 3.14.7 — documentación oficial, “Built-in Functions”. La referencia oficial está organizada alfabéticamente y contiene las firmas, excepciones, ejemplos y cambios de versión de cada built-in.

<https://docs.python.org/es/3.14/library/functions.html>